produces:

```
value
Another value
```

The second form here is the rightward assignment we've already seen. This can be used in more complex patterns:

```
[1, "potato", 2, "potato"] => [first, String, second, String]

puts "the numbers are #{first} and #{second}"
```

produces:

```
the numbers are 1 and 2
```

There's a shorter shortcut for hash patterns where you are only asserting that the key exists, including the name of the key assigned a local variable with that name.

```
{rank: "Ace", suit: "Hearts"} => {rank:, suit:}

puts "Your card is the #{rank} of #{suit}."
```

produces:

```
Your card is the Ace of Hearts.
```

If the pattern doesn't match, the variable assignment behavior is technically "undefined" to allow for potential performance improvements in the future. It looks like variables are assigned up to the point of the first mismatch, but we wouldn't recommend depending on that. This pattern assigns first but not second:

```
[1, "potato", 2, "potato"] in [Integer => first, Integer, Integer => second, String]

puts "the numbers are #{first} and #{second}"
```

produces:

```
the numbers are 1 and
```

If you use `=>` instead of `in` here, the behavior is much more clearly defined, you get a `NoMatchingPatternError` error and neither variable is assigned.

There are two limitations on assigning variables in pattern matching. First, you can only assign to a local variable. Specifically, you cannot assign to an `@` instance variable inside a pattern match. This seems to be related to performance and thread-safety concerns (it's actually related to the undefined performance of failed matches mentioned above), and there's a decent chance this changes in the future.

And second, you cannot do a variable assignment inside a pattern that uses the `|` to provide alternative patterns, you'll get a syntax error. We're honestly not 100% sure why this is, but we suspect it's related to the performance concerns.

Case Pattern Matching

Having mentioned that pattern matching uses the Ruby `===` operator, and that it compares a target value against another value, this might remind you of Ruby's `case` statement case Expressions, on page 154, which also uses the `===` operator and compares a target value.

And in fact, Ruby does support a `case/in` statement that pattern matches the target against one or more successive patterns:

```
def pick_a_card(cards)
```